

Game-Based Coding Problems

Board games, puzzles, and mobile-game mechanics — and the algorithms hiding inside them

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

Why games make great interview problems

Game questions are a favorite because they're **easy to explain** ("you're on a Snakes & Ladders board...") yet force you to do the two things interviewers most want to see:

1. **Model the state.** What is a "position"? What is a "move"? What is the "goal"? Getting this right is 80% of the solution.
2. **Pick the traversal/DP.** Once the state is modeled, most games collapse into a handful of patterns: **BFS over states** (shortest moves), **backtracking** (place/try/undo), **interval DP** (burst/remove sequences), **game-theory minimax DP** (optimal two-player play), or **simulation** (just play it out with the right data structure).

Evidence tags

Every problem here maps to a **real, well-known LeetCode problem**, so each carries a factual tag:

[LEETCODE — LC ###] — this is a genuine LeetCode problem at the given number. The "game" framing *is* the problem's natural framing (it really is about Snakes & Ladders, a knight, etc.), so there's no disguise to see through — but recognizing the underlying pattern is still the whole game.

Where a problem is a common variant or a mobile-game-flavored extension, that's noted inline.

The state-modeling checklist (use it on every game problem)

1. **STATE** : what fully describes a game position?
(a cell? a board config? a lock combination? whose turn?)
2. **MOVE** : from one state, what states can I reach in one step?
3. **GOAL** : which state(s) am I trying to reach / evaluate?
4. **PATTERN:**
 - shortest # of moves → BFS over states
 - "try all placements" → backtracking (place → recurse → undo)
 - burst/remove a sequence → interval DP (think last-to-burst)
 - optimal two-player play → minimax DP (maximize my score – yours)
 - just run the rules → simulation (queue / stack / grid)

The 16 problems

#	Game	Pattern	Tag
1	Snakes & Ladders	BFS on a flattened board	LC 909
2	Knight's Minimum Moves	BFS on an infinite board	LC 1197
3	N-Queens	backtracking	LC 51
4	Sudoku Solver	backtracking + constraints	LC 37
5	Word Search (Boggle)	grid DFS / backtracking	LC 79
6	Burst Balloons	interval DP (last to burst)	LC 312
7	Remove Boxes	interval DP (3D state)	LC 546
8	Candy Crush	grid simulation + marking	LC 723
9	Design Snake Game	queue + hashset simulation	LC 353
10	Sliding Puzzle	BFS over board states	LC 773
11	Open the Lock	BFS over states + dead-ends	LC 752
12	Stone Game	game-theory / math	LC 877
13	Predict the Winner	minimax DP	LC 486
14	Frog Jump	DP over (stone, jump) states	LC 403
15	Ability Cooldown Scheduler	greedy + heap	LC 621
16	Design Leaderboard	heap / TreeMap design	LC 1244

1. Snakes & Ladders

Evidence: [LEETCODE — LC 909, "Snakes and Ladders"]

How the interviewer poses it. "You're given an $n \times n$ board for Snakes & Ladders. The squares are numbered 1 to n^2 in a **boustrophedon** (boustrophedon = ox-plow) pattern: starting at the bottom-left square, numbers go left-to-right across the bottom row, then right-to-left across the row above it, alternating all the way to the top. From your current square you roll a die and can move forward **1 to 6** squares. If the square you land on has a snake or a ladder, you **immediately** move to the square it points to (you never take a second snake/ladder on the same turn). Starting at square 1, what is the **minimum number of dice rolls** to reach square n^2 ? Return -1 if it's impossible."

De-story it

Strip the board-game nouns:

```
nouns to ignore: "die", "snake", "ladder", "boustrophedon"
INPUT    : an n x n grid `board` where board[r][c] is either -1 (no
           effect) or the destination square number (a snake/ladder)
OUTPUT   : min number of moves from square 1 to square  $n^2$ , where
           each move advances the current square by 1..6, then
           optionally jumps via the destination at the landing square
CONSTRAINT: -1 if  $n^2$  is unreachable
```

That is **shortest path in an unweighted graph**: each square is a node, and square s has an edge to squares $s+1 \dots s+6$ (or to their snake/ladder destinations). "Fewest moves" on an unweighted graph is the textbook signal for **BFS**.

Worked example

$n = 4$, squares 1..16, laid out boustrophedon (bottom row goes right, each row above alternates direction):

columns:	c0	c1	c2	c3
row0 (top):	16	15	14	13
row1:	9	10	11	12
row2:	8	7	6	5
row3 (bottom):	1	2	3	4

one ladder: square 2 $\rightarrow$ 15 (board[row3][c1] = 15)

one snake : square 13 $\rightarrow$ 3 (board[row0][c3] = 3)

everything else is -1

from square 1: roll 1 $\rightarrow$ land on 2 $\rightarrow$ ladder $\rightarrow$ 15

from square 15: roll 1 $\rightarrow$ land on 16 = target

Answer: 2 rolls (no other path reaches 16 in fewer moves – the next best from square 1 without the ladder tops out around square 13)

Intuition (diagram)

1 --roll1--> 2 =ladder=> 15 --roll1--> 16 (target)
 \--roll2..6--> 3,4,5,6,7 (all dead-end at ≤ 2 rolls to 16)

Approach

Two things make this more than "plain BFS":

1. **Flatten the board.** The board is stored as a 2D grid in *display* order (row 0 = top row, last row = bottom), but squares are numbered in *boustrophedon* order starting at the bottom-left. So square `s` does **not** map to `board[s / n][s % n]` — you have to reconstruct which matrix row/column it lives in, respecting the alternating direction.
2. **Apply the snake/ladder before enqueueing.** Landing on square `s+move` may immediately redirect you to `board[row][col]`; the graph edge goes from the *current* square to the *post-jump* square, not the raw landing square.

Flattening formula, given 0-indexed `id = square - 1`: - `rowFromBottom = id / n` — 0 for the bottom row, 1 for the next row up, etc. - `colInRow = id % n` — position within that row, always counted left-to-right in *reading* order of the numbering. - If `rowFromBottom` is even, the row was numbered left-to-right, so `col = colInRow`. If odd, the row was numbered right-to-left, so `col = n - 1 - colInRow`. - The matrix row index (top = 0) is `n - 1 - rowFromBottom`, since the bottom row (`rowFromBottom = 0`) is the *last* row of the matrix.

Once you have `(row, col)`, `board[row][col]` is either `-1` (stay on square) or the snake/ladder destination.

Brute force

Try every dice sequence with DFS, backtracking, and keep the best rolls count found so far. It works, but it re-explores huge amounts of shared structure (many sequences pass through the same square) and has no polynomial bound.

```
private int best = Integer.MAX_VALUE;

public int minRollsBrute(int[][] board, int square, int target, int rolls, Set<Integer> onPath) {
    if (square == target) {
        best = Math.min(best, rolls);
        return best;
    }
    if (rolls >= best) return best; // can't possibly beat the best known
    for (int move = 1; move <= 6 && square + move <= target; move++) {
        int next = valueAt(board, square + move); // land, then follow snake/ladder
        if (onPath.add(next)) { // avoid cycling forever on this path
            minRollsBrute(board, next, target, rolls + 1, onPath);
            onPath.remove(next);
        }
    }
    return best;
}
```

Optimized (BFS on the flattened board)

Treat squares $1..n^2$ as nodes of an unweighted graph and BFS from square 1. The first time you dequeue the target, `dist[target]` is the answer — BFS guarantees the first visit to any node is via a shortest path.

```

public int snakesAndLadders(int[][] board) {
    int n = board.length;
    int target = n * n;
    int[] dist = new int[target + 1];
    Arrays.fill(dist, -1); // -1 = unvisited
    dist[1] = 0;

    Queue<Integer> queue = new LinkedList<>();
    queue.offer(1);

    while (!queue.isEmpty()) {
        int cur = queue.poll();
        if (cur == target) return dist[cur];
        for (int move = 1; move <= 6 && cur + move <= target; move++) {
            int next = valueAt(board, cur + move); // apply snake/ladder, if any
            if (dist[next] == -1) {
                dist[next] = dist[cur] + 1;
                queue.offer(next);
            }
        }
    }
    return -1; // target never reached
}

private int valueAt(int[][] board, int square) {
    int n = board.length;
    int id = square - 1; // 0-indexed square
    int rowFromBottom = id / n;
    int colInRow = id % n;
    int col = (rowFromBottom % 2 == 0) ? colInRow : n - 1 - colInRow;
    int row = n - 1 - rowFromBottom; // matrix row 0 is the top row
    int value = board[row][col];
    return value == -1 ? square : value; // -1 means "stay here"
}

```

Version	Time	Space
Brute force (DFS over all dice sequences)	$O(6^{(n^2)})$ worst case	$O(n^2)$ recursion/path
BFS on flattened board	$O(n^2)$	$O(n^2)$

What to say

"This is shortest path on an unweighted graph: square `s` has an edge to `s+1..s+6`, redirected through any snake or ladder at the landing square, so fewest moves means BFS from square 1 and return the distance to `n^2`. The one trick is the board isn't indexed the way it's numbered — the numbering snakes bottom-to-top with alternating direction, so I convert a square number to a `(row, col)` by peeling off `rowFromBottom` and `colInRow`, flipping the column when the row was numbered right-to-left. I apply the snake/ladder redirect *before* marking a square visited, since the edge really goes to the post-jump square. That's $O(n^2)$ time and space instead of the brute force's exponential dice-sequence search."

2. Knight's Minimum Moves

Evidence: [LEETCODE — LC 1197, "Minimum Knight Moves"]

How the interviewer poses it. "On an infinite chessboard, a knight starts at square `(0, 0)`. It moves the way a chess knight does — an L-shape, two squares in one direction and one square perpendicular to that. Given a target square `(x, y)`, return the **minimum number of moves** the knight needs to reach it."

De-story it

Strip the chess nouns:

```
nouns to ignore: "knight", "chessboard", "square"
INPUT    : a start node (0,0) and a target node (x, y) on an implicit,
           unbounded grid graph
OUTPUT   : the minimum number of edges to get from start to target
CONSTRAINT: each node has exactly 8 outgoing edges (the 8 knight jumps);
           the grid is infinite in all directions
```

That is **shortest path on an unweighted graph** — every edge costs 1 move — which is the textbook signal for **BFS**. The only twist is that the graph isn't given as an adjacency list; you generate a node's 8 neighbors on the fly with a fixed offset table.

Worked example

```
target = (2, 1)

BFS layers from (0,0):
  layer 0: (0,0)
  layer 1: (1,2) (2,1) (2,-1) (1,-2) (-1,-2) (-2,-1) (-2,1) (-1,2)
           -> (2,1) is already in layer 1!
Answer: 1 move (one direct knight jump reaches (2,1))

target = (5, 5)
  layer 0: (0,0)
  layer 1: 8 squares, none is (5,5)
  layer 2: up to 8*8 squares (with overlap/dedupe), none is (5,5)
  layer 3: (5,5) first appears here
Answer: 3 moves
```

Intuition (diagram)

. N . N .	N = the 8 landing spots one
N . . . N	knight-jump away from the
. . K . .	knight K in the center
N . . . N	
. N . N .	

Each **N** is one BFS neighbor of **K**. Treat the board as a graph, **K** as the source, and expand outward one knight-jump at a time — first time you touch the target, that layer number is the answer.

Approach: BFS with symmetry pruning

Naively, BFS on an infinite board looks unbounded, but two observations tame it:

1. **Symmetry.** The knight-move graph is symmetric across both axes and the diagonal, so the number of moves to (x, y) equals the number of moves to $(|x|, |y|)$. Reflecting the target into the first quadrant shrinks the search space by up to 4x and avoids exploring three mirror-image copies of the same work.
2. **Bounding.** Once we've reflected into the first quadrant, the shortest path never needs to wander far outside the bounding box of the start and target — allowing the knight a small margin (e.g., 2 squares) past 0 in either axis is enough slack to "wind up" for a jump without ever needing to explore arbitrarily far negative coordinates.


```

public int minKnightMoves(int x, int y) {
    x = Math.abs(x);           // exploit symmetry: fold into first quadrant
    y = Math.abs(y);

    int[][] dirs = {
        {1, 2}, {2, 1}, {2, -1}, {1, -2},
        {-1, -2}, {-2, -1}, {-2, 1}, {-1, 2}
    };

    Queue<int[]> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    queue.offer(new int[]{0, 0});
    visited.add("0,0");

    int moves = 0;
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] cur = queue.poll();
            if (cur[0] == x && cur[1] == y) return moves;    // reached target
            for (int[] d : dirs) {
                int nx = cur[0] + d[0];
                int ny = cur[1] + d[1];
                if (nx < -2 || ny < -2) continue;          // small margin past 0
                String key = nx + "," + ny;
                if (visited.add(key)) {                     // add() returns false if seen
                    queue.offer(new int[]{nx, ny});
                }
            }
        }
        moves++;
    }
    return -1;                                             // unreachable (never happens here)
}

```

Optimized (bidirectional BFS)

Same BFS, but grow a frontier from `(0,0)` **and** a frontier from `(x, y)` at the same time, swapping to whichever frontier is smaller each round, and stop the moment the two frontiers touch. This halves the effective search radius, which matters once `x, y` get large.

```

public int minKnightMovesBidirectional(int x, int y) {
    x = Math.abs(x);
    y = Math.abs(y);
    int[][] dirs = {
        {1, 2}, {2, 1}, {2, -1}, {1, -2},
        {-1, -2}, {-2, -1}, {-2, 1}, {-1, 2}
    };

    Set<String> visitedFromStart = new HashSet<>();
    Set<String> visitedFromTarget = new HashSet<>();
    Set<String> frontierStart = new HashSet<>();
    Set<String> frontierTarget = new HashSet<>();
    frontierStart.add("0,0");
    frontierTarget.add(x + "," + y);
    visitedFromStart.add("0,0");
    visitedFromTarget.add(x + "," + y);

    int moves = 0;
    while (!frontierStart.isEmpty() && !frontierTarget.isEmpty()) {
        if (!Collections.disjoint(frontierStart, frontierTarget)) return moves;

        // always expand the smaller frontier
        if (frontierStart.size() > frontierTarget.size()) {
            Set<String> tmp = frontierStart; frontierStart = frontierTarget; frontierTarget = tmp;
            Set<String> tmp2 = visitedFromStart; visitedFromStart = visitedFromTarget; visitedFromTarget = tmp2;
        }

        Set<String> next = new HashSet<>();
        for (String node : frontierStart) {
            String[] parts = node.split(",");
            int cx = Integer.parseInt(parts[0]);
            int cy = Integer.parseInt(parts[1]);
            for (int[] d : dirs) {
                int nx = cx + d[0];
                int ny = cy + d[1];
                if (nx < -2 || ny < -2) continue;
                String key = nx + "," + ny;
                if (visitedFromStart.add(key)) next.add(key);
            }
        }
        frontierStart = next;
        moves++;
    }
    return -1;
}

```

Version	Time	Space
BFS with symmetry fold	$O(\max(x, y)^2)$	$O(\max(x, y)^2)$
Bidirectional BFS	$O(\max(x, y)^2)$ worst case, far less in practice	$O(\max(x, y)^2)$

If the interviewer pushes further: there's a closed-form/DP approximation based on Manhattan distance and parity ($\text{delta} = \max(dx, dy)$, adjust for the knight's 2-square reach), but it's fiddly to get exactly right — mention it exists as the "O(1) math" answer, then default back to BFS as the reliable, provably-correct approach.

What to say

"Ignoring the chess framing, this is shortest path on an unweighted graph where each node has 8 fixed-offset neighbors, so BFS is the natural fit. I'll fold the target into the first quadrant using the knight-move graph's symmetry to cut the search space, bound the frontier with a small negative margin so it doesn't wander needlessly, and expand layer by layer until I hit the target. If the coordinates are huge, I'd switch to bidirectional BFS to shrink the effective radius."

3. N-Queens

Evidence: [LEETCODE — LC 51, "N-Queens"]

How the interviewer poses it. "Place `n` chess queens on an `n x n` board so that **no two queens attack each other** — meaning no two share a row, a column, or a diagonal. Return **all distinct ways** to place the queens, each as a board configuration."

De-story it

Strip the chess nouns:

```
nouns to ignore: "queen", "chessboard", "attack"
INPUT   : an integer n (board size)
OUTPUT  : all placements of n markers, one per row, such that no two
          markers share a column or a diagonal
CONSTRAINT: exactly one marker per row and per column; no two markers
          on the same diagonal
```

That is a **constraint-satisfaction placement** problem: fill in choices one row at a time, and only keep choices that don't violate a growing set of constraints → **classic backtracking**, one decision (which column) per row.

Worked example

```
n = 4

place row 0 -> try col 0: ok so far
  place row 1 -> col 0,1 blocked (col/diag) -> try col 2: ok
    place row 2 -> col 0,1,2,3 all blocked by row0/row1 -> dead end, backtrack
  place row 1 -> try col 3: ok
    place row 2 -> try col 1: ok
      place row 3 -> col 0,1,2,3 all blocked -> dead end, backtrack all the way

... continuing the search finds 2 valid boards for n = 4:
.Q..      ..Q.
...Q      Q...
Q...      ...Q
..Q.      .Q..
```

Intuition (diagram)

Think of it as filling the board row by row; each row picks exactly one free column.

```
row 0: . Q . .      col used:      1
row 1: . . . Q      diag(r-c) used: -1, -3
row 2: Q . . .      diag(r+c) used:  1,  3
row 3: . . Q .      <- must dodge every used col/diagonal
```

Approach (backtracking)

Go row by row. For the current row, try every column; a column is legal only if it isn't already used by an earlier queen's **column**, its **"/" diagonal** (`row - col`), or its **"\" diagonal** (`row + col`). Place the queen, recurse into the next row, then undo (remove the queen and free the column/diagonals) before trying the next column.

The brute-force alternative — trying every one of the $C(n^2, n)$ ways to drop n queens on n^2 squares and checking each for conflicts — is astronomically larger, so backtracking (which prunes a branch the instant a placement is invalid) is the only practical solution; there isn't a separate "optimized" pass to layer on top of it.

```

public List<List<String>> solveNQueens(int n) {
    List<List<String>> results = new ArrayList<>();
    int[] queenCol = new int[n];           // queenCol[row] = column of the queen in that row
    Set<Integer> cols = new HashSet<>();
    Set<Integer> diag1 = new HashSet<>();   // row - col
    Set<Integer> diag2 = new HashSet<>();   // row + col

    backtrack(0, n, queenCol, cols, diag1, diag2, results);
    return results;
}

private void backtrack(int row, int n, int[] queenCol, Set<Integer> cols,
    Set<Integer> diag1, Set<Integer> diag2,
    List<List<String>> results) {
    if (row == n) {                        // every row has a queen -> valid board
        results.add(buildBoard(queenCol, n));
        return;
    }
    for (int col = 0; col < n; col++) {
        int d1 = row - col, d2 = row + col;
        if (cols.contains(col) || diag1.contains(d1) || diag2.contains(d2)) {
            continue;                      // column or a diagonal is already taken
        }
        queenCol[row] = col;               // choice: place queen at (row, col)
        cols.add(col); diag1.add(d1); diag2.add(d2);

        backtrack(row + 1, n, queenCol, cols, diag1, diag2, results);

        cols.remove(col); diag1.remove(d1); diag2.remove(d2); // undo
    }
}

private List<String> buildBoard(int[] queenCol, int n) {
    List<String> board = new ArrayList<>();
    for (int row = 0; row < n; row++) {
        char[] line = new char[n];
        Arrays.fill(line, '.');
        line[queenCol[row]] = 'Q';
        board.add(new String(line));
    }
    return board;
}

```

Version	Time	Space
Brute force (all $C(n^2, n)$ placements)	astronomically worse than $O(n!)$	$O(n)$
Backtracking + pruning (row-by-row, set checks)	$O(n!)$ worst case, far less in practice	$O(n)$ sets + recursion

If the interviewer asks for the count only (N-Queens II): skip building board strings and just increment a counter at `row == n`; same backtracking, less bookkeeping.

What to say

"Ignoring the chess framing, this is constraint-satisfaction placement: one queen per row, and each choice must avoid the columns and diagonals already claimed. I'll backtrack row

by row, track used columns and both diagonals with sets, place a queen, recurse, and undo on the way back. Brute-forcing every placement of n queens on n^2 squares is astronomically larger, so pruning invalid columns as I go is what makes this tractable, roughly $O(n!)$ in the worst case."

4. Sudoku Solver

Evidence: [LEETCODE — LC 37, "Sudoku Solver"]

How the interviewer poses it. "Here's a 9x9 Sudoku board. Some cells already hold a digit 1-9, and the rest are blank (shown as `.`). **Fill in the blanks** so that every row, every column, and every one of the nine 3x3 boxes contains the digits **1 through 9 exactly once**. Assume the input has exactly one solution — just find it."

De-story it

Strip the puzzle-box framing:

```
nouns to ignore: "board", "cell", "box"
INPUT   : a 9x9 grid of chars, each '1'-'9' or '.' (blank)
OUTPUT  : the same grid with every '.' replaced by a digit
CONSTRAINT: each row, column, and 3x3 box must contain 1-9
           with no repeats, once filled
```

That is **constraint satisfaction**: assign values to unknowns one at a time, subject to row/column/box rules. Whenever a problem says "fill in blanks so constraints hold" → **backtracking**: pick an empty slot, try each legal value, recurse, and undo if a later slot has no legal value left.

Worked example

```
a 4x4 mini-Sudoku (digits 1-4, 2x2 boxes) to keep it small:
```

```
1 . | . 4
. 4 | 1 .
-----
. 1 | 4 .
4 . | . 1
```

```
empty cell (0,1): row has {1,4}, col has {1,4}, box has {1,4}
-> only 2 or 3 are legal; try 2, recurse; if that path dead-ends,
    undo and try 3
the solver keeps doing this cell by cell until the grid is full
or every choice at some cell fails, forcing backtrack further up
```

Intuition (diagram)

Each empty cell branches into up to 9 tries; box = 3x3 sub-grid.


```

row -> [5][3][.][.][7][.][.][.][.]
       [6][.][.][1][9][5][.][.][.]
       [.] [9][8][.][.][.][.][6][.]
       -----+-----+-----
       [8][.][.][.][6][.][.][.][3]
       ^box: the 3x3 block this cell belongs to
pick "." -> try 1..9 -> check row, col, box -> place or backtrack

```

Approach (backtracking)

Pure brute force would try all 9 digits in every empty cell independent of validity — that's $9^{(\text{empty cells})}$ fillings before even checking constraints. Instead, prune as we go: only try digits that don't already violate a row, column, or box, and stop (undo) the instant a branch has no legal digit.

```

public void solveSudoku(char[][] board) {
    solve(board);
}

private boolean solve(char[][] board) {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (board[r][c] != '.') continue; // find next empty cell

            for (char d = '1'; d <= '9'; d++) {
                if (!isValid(board, r, c, d)) continue;

                board[r][c] = d; // choice: place d
                if (solve(board)) return true; // recurse on rest
                board[r][c] = '.'; // undo: doesn't lead anywhere
            }
            return false; // no digit works here -> backtrack
        }
    }
    return true; // no empty cells left: solved
}

private boolean isValid(char[][] board, int r, int c, char d) {
    int boxRow = (r / 3) * 3, boxCol = (c / 3) * 3;
    for (int i = 0; i < 9; i++) {
        if (board[r][i] == d) return false; // row has d
        if (board[i][c] == d) return false; // col has d
        if (board[boxRow + i / 3][boxCol + i % 3] == d) return false; // box has d
    }
    return true;
}

```

Faster validity checks: instead of rescanning 9 cells each time, keep a `boolean[9][9]` (or `int bitmask`) per row, per column, and per box marking which digits are already used. Placing/undoing a digit flips one bit, and `isValid` becomes $O(1)$ instead of $O(9)$.

Version	Time	Space
Brute force (try all 9 digits everywhere)	$O(9^{(\text{empty cells})})$	$O(1)$ extra

Version	Time	Space
Backtracking + row/col/box check	$O(9^{\text{empty cells}})$ worst case, far less in practice	$O(1)$ extra ($O(81)$ recursion)
Backtracking + bitmask sets	same worst case, faster constant per call	$O(1)$ extra for the masks

If the interviewer pushes for speed: mention picking the empty cell with the **fewest remaining legal digits** first (minimum-remaining-values heuristic) — it fails fast on dead branches and prunes the tree far more aggressively than scanning left-to-right.

What to say

"This is constraint satisfaction, not really a 'puzzle' problem — I fill empty cells one at a time, and for each I only try digits that don't already appear in that row, column, or 3x3 box. If a cell has no legal digit, I undo the last placement and try the next option there. Worst case is exponential in the number of empty cells, but the row/col/box pruning — and bitmask sets if I want it faster — keeps it fast in practice."

5. Word Search (Boggle)

Evidence: [LEETCODE — LC 79, "Word Search"]

How the interviewer poses it. "You're given an `m x n` grid of single letters and a target `word`. Return `true` if `word` can be built by starting at some cell and moving to horizontally or vertically adjacent cells, one step at a time — but you may **not reuse the same cell** twice within one word."

De-story it

Strip the board-game framing:

```
nouns to ignore: "grid", "cell", "board"
INPUT   : a 2D char array `board`, and a string `word`
OUTPUT  : true if `word` exists as a path of 4-directionally adjacent
          cells (up/down/left/right), false otherwise
CONSTRAINT: no cell may be reused within a single path
```

That is a **path-existence search on a grid**: try every starting cell, and from each one walk the grid trying to match the word letter by letter. A cell used earlier in the path is unavailable — the classic setup for **DFS with backtracking**: mark a cell visited before recursing into its neighbors, then unmark it when the recursion returns so other paths can use it again.

Worked example

```
board = [['A','B','C','E'],
         ['S','F','C','S'],
         ['A','D','E','E']]
word  = "ABCCED"

try start (0,0)='A' matches word[0]
  -> (0,1)='B' matches word[1]
    -> (0,2)='C' matches word[2]
      -> (1,2)='C' matches word[3]
        -> (2,2)='E' matches word[4]
          -> (2,1)='D' matches word[5] -- word fully matched
Answer: true
```

Intuition (diagram)

```
[A]->[B]->[C]  E
S      F   [C]  S
A      [D]<-[E]  E
```

The path snakes right along row 0, drops down a row, then doubles back left — each bracketed cell is "used" for this attempt only, and gets freed the instant that branch fails.

Approach (DFS backtracking)

```
public boolean exist(char[][] board, String word) {
    int rows = board.length, cols = board[0].length;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (dfs(board, r, c, word, 0)) return true;    // try every start cell
        }
    }
    return false;
}

private boolean dfs(char[][] board, int r, int c, String word, int idx) {
    if (idx == word.length()) return true;                // matched every letter
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
    if (board[r][c] != word.charAt(idx)) return false;

    char saved = board[r][c];
    board[r][c] = '#';                                    // mark this cell "in use" for the path

    boolean found = dfs(board, r + 1, c, word, idx + 1)
        || dfs(board, r - 1, c, word, idx + 1)
        || dfs(board, r, c + 1, word, idx + 1)
        || dfs(board, r, c - 1, word, idx + 1);

    board[r][c] = saved;                                    // unmark: backtrack so other paths can reuse it
    return found;
}
```

If asked to search many words at once (LC 212, Word Search II): don't repeat this DFS per word.

Build a **Trie** of all target words first, then run one grid-wide DFS that walks the Trie alongside the board, so shared prefixes are explored once instead of once per word.

Metric	Value
Time	$O(m \cdot n \cdot 4^L)$ — $m \cdot n$ start cells, up to 4 branches per letter of length L
Space	$O(L)$ recursion depth (plus the $O(1)$ in-place cell marking)

What to say

"Ignoring the board-game framing, this is path existence on a grid: from each starting cell, DFS through 4-directional neighbors trying to match the word letter by letter. I'll mark a cell visited before recursing and restore it on the way back out so backtracking can reuse it on a different path. If I needed to search many words at once, I'd build a Trie of the words and walk it alongside one shared DFS instead of repeating this per word."

6. Burst Balloons

Evidence: [LEETCODE — LC 312, "Burst Balloons" (Hard)]

How the interviewer poses it. "You're given `n` balloons lined up in a row, each with a number painted on it, `nums[i]`. If you burst balloon `i`, you collect `nums[left] * nums[i] * nums[right]` coins, where `left` and `right` are the balloons currently **adjacent** to `i` (treat a missing left/right end as a balloon with value `1`). Once balloon `i` is burst, its former neighbors become adjacent to each other. Burst all the balloons in whatever order you like. **What is the maximum number of coins you can collect?**"

De-story it

Strip the carnival-game nouns:

```
nouns to ignore: "balloon", "burst", "painted number"
INPUT   : an array of positive integers nums[0..n-1] (the balloon values)
OUTPUT  : the maximum total coins obtainable by removing every element,
          one at a time, where removing nums[i] scores
          nums[left] * nums[i] * nums[right] using whatever elements
          are CURRENTLY adjacent to i
CONSTRAINT: the score for each removal depends on the removal ORDER,
            not just on which elements remain
```

That is an **order-of-operations optimization**: the array shrinks as you go, and the payoff for each step depends on the current neighbors. Whenever the *sequence of removals* — not just the final set of removals — changes the total, that's the signature of **interval DP**.

Worked example

```
nums = [3, 1, 5, 8]
pad with 1's at both ends: [1, 3, 1, 5, 8, 1]   (indices 0..5)

one burst order: burst 1 (value 1), then 2 (value 5), then 0 (value 3), then 3 (value 8)
burst index1(1): neighbors 3,5   -> 3*1*5   = 15
burst index2(5): neighbors 3,8   -> 3*5*8   = 120
burst index0(3): neighbors 1,8   -> 1*3*8   = 24
burst index3(8): neighbors 1,1   -> 1*8*1   = 8
total = 15 + 120 + 24 + 8 = 167  <-- optimal
```

Intuition (diagram)

The trick: for any range `(i, j)`, think about which balloon is burst **LAST** inside that range. Call it `k`. Since every other balloon in `(i, j)` is already gone, `k`'s neighbors AT THAT MOMENT are exactly the

fixed boundary balloons i and j — so its score is $\text{nums}[i] * \text{nums}[k] * \text{nums}[j]$, always. The two sides of k never interact with each other, so they can be solved independently and added.

```

range (i ..... j)
      i [ left part ] k [ right part ] j
      dp[i][k]      dp[k][j]

k = LAST balloon burst in (i,j)
  -> at that instant, only i and j remain as neighbors
  -> score(k) = nums[i] * nums[k] * nums[j]    (fixed! doesn't depend
                                              on order inside each side)

dp[i][j] = max over k in (i,j) of:
          nums[i]*nums[k]*nums[j] + dp[i][k] + dp[k][j]

```

This reframing turns a messy "which order?" search into a clean interval recurrence: pick the *last* balloon in the range, and the two sub-ranges it creates are independent subproblems.

Brute force

Try every possible burst order directly (recursion + memo-free backtracking over permutations, recomputing neighbors after each burst) — roughly $n!$ orderings.

```

public int maxCoinsBrute(int[] nums) {
    int n = nums.length;
    int[] padded = new int[n + 2];
    padded[0] = padded[n + 1] = 1;
    for (int i = 0; i < n; i++) padded[i + 1] = nums[i];
    List<Integer> balloons = new ArrayList<>();
    for (int v : padded) balloons.add(v);
    return burstAll(balloons);
}

// balloons still includes the two boundary 1's at index 0 and last index
private int burstAll(List<Integer> balloons) {
    if (balloons.size() == 2) return 0;           // only the two 1-pads left
    int best = 0;
    for (int i = 1; i < balloons.size() - 1; i++) { // try every real balloon next
        int left = balloons.get(i - 1);
        int mid = balloons.get(i);
        int right = balloons.get(i + 1);
        int gained = left * mid * right;
        List<Integer> next = new ArrayList<>(balloons);
        next.remove(i);                          // burst it, neighbors close up
        best = Math.max(best, gained + burstAll(next));
    }
    return best;
}

```

Optimized (bottom-up interval DP)

Pad `nums` with a 1 on each end. $\text{dp}[i][j]$ = max coins from bursting every balloon strictly between padded indices i and j , leaving i and j themselves intact as boundaries. Build up by increasing gap length so $\text{dp}[i][k]$ and $\text{dp}[k][j]$ are already known before $\text{dp}[i][j]$ needs them.

```

public int maxCoins(int[] nums) {
    int n = nums.length;
    int[] p = new int[n + 2];
    p[0] = p[n + 1] = 1;
    for (int i = 0; i < n; i++) p[i + 1] = nums[i];

    int[][] dp = new int[n + 2][n + 2];
    for (int len = 2; len <= n + 1; len++) {           // gap width between i and j
        for (int i = 0; i + len <= n + 1; i++) {
            int j = i + len;
            for (int k = i + 1; k < j; k++) {           // k = LAST balloon burst in (i,j)
                dp[i][j] = Math.max(dp[i][j],
                    p[i] * p[k] * p[j] + dp[i][k] + dp[k][j]);
            }
        }
    }
    return dp[0][n + 1];
}

```

Version	Time	Space
Brute force (every burst order)	$O(n!)$	$O(n)$ recursion + copies
Bottom-up interval DP	$O(n^3)$	$O(n^2)$

What to say

"The payoff for bursting a balloon depends on the order, since neighbors change as balloons disappear — that's the tell for interval DP. The key move is to stop asking 'which balloon do I burst first?' and instead ask 'which balloon in this range is burst LAST?' Whatever that balloon k is, its neighbors at that moment are the fixed range boundaries i and j , so its score is just $nums[i] * nums[k] * nums[j]$, and the two sides of k become independent subproblems. I pad the array with 1's on both ends, define $dp[i][j]$ over the open interval between i and j , and build up by increasing interval length for $O(n^3)$ total."

7. Remove Boxes

Evidence: [LEETCODE — LC 546, "Remove Boxes" (Hard)]

How the interviewer poses it. "You've got a row of boxes on a shelf, each painted one of several colors. You may remove any **contiguous run of boxes that are all the same color**; if you remove k boxes in one move, you score $k * k$ points. After a removal, the boxes on either side slide together to close the gap — so colors that used to be far apart can end up touching. Choose the order of removals that **clears the whole shelf** while maximizing your total score."

De-story it

Strip the shelf/paint nouns:

```
nouns to ignore: "boxes", "shelf", "paint"
INPUT    : an array of integers, each value is a "color id"
OUTPUT   : the maximum total score obtainable by repeatedly removing
           contiguous runs of equal values, where removing a run of
           length  $k$  scores  $k*k$  points, until the array is empty
CONSTRAINT: a run must be CONTIGUOUS and all one value at the moment
           you remove it; after a removal the remaining values slide
           together, so a removal can create BRAND NEW same-color
           runs that didn't exist before
```

That last clause is the whole problem. A plain interval DP over $dp[i][j]$ ("best score for the subarray $i..j$ ") is not enough, because the best play often means **not** removing a same-colored box the instant you see it — you hold it, clear out what's between it and a matching color further right, let them slide together, and cash in the bigger combined run later. You need a third dimension that remembers "how many same-colored boxes are already waiting, glued to the left edge of this interval" — call it k . That gives $dp[i][j][k]$.

Worked example

```
boxes: 1 3 2 2 2 3 4 3 1

move 1: remove the run of three 2's      (k=3) -> score 9
        left over, after sliding together:
        1 3 3 4 3 1

move 2: remove the lone 4                (k=1) -> score 1
        left over, after sliding together:
        1 3 3 3 1

move 3: remove the run of three 3's      (k=3) -> score 9
        (this run only EXISTS because moves 1-2 let the
        two original 3's slide next to the third 3)
        left over:
        1 1

move 4: remove the run of two 1's        (k=2) -> score 4

total: 9 + 1 + 9 + 4 = 23
```

Notice move 3's run of three 3's never appears in the original array — it was manufactured by removing the 2's and the 4 out of the way first. That's the behavior the DP has to plan for.

Intuition (diagram)

```
dp[i][j][k] = best score for boxes[i..j], given that k
              boxes of boxes[i]'s color are already
              glued onto i's left edge, waiting

choice A: pop box i (+ its k glued buddies) right now
          score = (k+1)^2 + dp[i+1][j][0]

choice B: leave box i waiting, clear the gap up to some
          later box m of the SAME color, let them touch
          score = dp[i+1][m-1][0] + dp[m][j][k+1]

answer = max(A, B over every valid m) = dp[0][n-1][0]
```

Approach (3D interval DP + memo)

Why not brute force? The naive plan is: try every legal order of removals (and every legal *split* of a run — you're even allowed to remove only part of a same-colored block and save the rest for later), recursing on whatever the shelf looks like after each move. Each removal can also manufacture brand-new same-color runs, which opens up moves that didn't exist a step ago. The number of distinct removal sequences explodes **factorially** (and worse, since merges keep creating new options) — it's only survivable for a handful of boxes, nowhere near an interview input size.

The DP tames this by never re-deriving the same sub-problem twice. State `dp[i][j][k]` covers the subarray `boxes[i..j]` plus a count `k` of same-colored boxes already "attached" to the left of `i` (conceptually already slid next to it, just not removed yet). From that state there are exactly two families of moves:

1. **Cash out now** — remove `boxes[i]` together with its `k` attached duplicates in one shot: scores $(k+1)^2$, then recurse on `dp[i+1][j][0]` (the attachment count resets, since we're starting a fresh interval).
2. **Wait and merge** — pick any later index `m` in `(i, j]` where `boxes[m] == boxes[i]`. Clear everything strictly between them optimally (`dp[i+1][m-1][0]`), let `i` and `m` slide together, and continue as `dp[m][j][k+1]` — now `m` carries `k+1` attached duplicates.

Take the max over choice 1 and every choice-2 candidate `m`, and memoize on `(i, j, k)` so each state is solved once.

Optimized (memoized interval DP)

```
class Solution {
    private int[][][] memo;
    private int[] boxes;

    public int removeBoxes(int[] boxes) {
        this.boxes = boxes;
        int n = boxes.length;
        memo = new int[n][n][n];          // memo[i][j][k], 0 means "not computed"
        return solve(0, n - 1, 0);
    }

    private int solve(int i, int j, int k) {
        if (i > j) return 0;
        if (memo[i][j][k] != 0) return memo[i][j][k];

        // slide any boxes right after i that already match its color into
        // the "attached" count, so i always sits at the true left edge
        int ii = i, kk = k;
        while (ii + 1 <= j && boxes[ii + 1] == boxes[i]) {
            ii++;
            kk++;
        }

        // choice 1: cash out box i (+ its kk attached duplicates) right now
        int best = (kk + 1) * (kk + 1) + solve(ii + 1, j, 0);

        // choice 2: leave i waiting, clear the middle, merge with a later
        // box m of the same color, then keep waiting as k+1 duplicates
        for (int m = ii + 1; m <= j; m++) {
            if (boxes[m] == boxes[i]) {
                best = Math.max(best, solve(ii + 1, m - 1, 0) + solve(m, j, kk + 1));
            }
        }

        memo[i][j][k] = best;
        return best;
    }
}
```

Version	Time	Space
Brute force (every removal order + run split)	factorial (and worse, from re-created runs)	$O(n)$ recursion
3D interval DP + memo	$O(n^4)$ — n^3 states, $O(n)$ work per state	$O(n^3)$ memo table

What to say

"Stripped of the shelf story, this is: repeatedly remove contiguous equal-value runs, scoring k^2 , and removals slide the array together so new runs can appear. A 2D interval DP on `dp[i][j]` can't capture 'hold this box and merge it with a matching one further right,' so I add a third dimension `k` for how many same-colored boxes are already waiting on the left edge. From `dp[i][j][k]` I either cash the left box out now for $(k+1)^2$, or clear the middle and merge it with a later same-colored box `m`, taking the max and memoizing on `(i, j, k)`. That's $O(n^4)$ instead of the factorial blowup of trying every removal order directly."

8. Candy Crush

Evidence: [LEETCODE — LC 723, "Candy Crush"]

How the interviewer poses it. "You're given a 2D grid of candies, where `board[r][c]` is a positive integer representing that candy's type (or `0` for an empty cell). Whenever **three or more candies of the same type** line up **horizontally or vertically**, they get **crushed** — turned to `0` — all at once. After crushing, candies **fall down** to fill the empty space below them (gravity), and new empty cells appear at the top. This crush-then-fall cycle **repeats** until no more matches exist. Return the final, stable board."

De-story it

Strip the candy-shop framing:

```
nouns to ignore: "candy", "board", "crush"
INPUT    : an m x n grid of ints; positive = a candy type, 0 = empty
OUTPUT   : the grid after repeatedly crushing runs of >=3 same-valued
           cells (row or column) and dropping remaining cells down,
           until one full pass finds nothing left to crush
CONSTRAINT: a "run" is 3+ adjacent cells, same value, same row OR
            same column; crushing is simultaneous per round, not
            one cell at a time; gravity acts per column, independently
```

That is **grid simulation**: no clever formula, just repeatedly (1) scan for matches, (2) zero them out, (3) apply gravity, and stop when a full scan finds no matches. The only subtlety is doing the *scan* for the whole board before doing any *crushing*, so a candy that's part of both a horizontal and a vertical run only gets crushed once, and matches don't shift mid-scan.

Worked example

```
before (4x4):      row1 has three 2's (cols 0-2)
1 2 3 4           col3 has four 4's (rows 0-3)
2 2 2 4
3 1 4 4
1 1 3 4

after marking + zeroing the two runs:
1 2 3 0
0 0 0 0
3 1 4 0
1 1 3 0

after gravity (each column's survivors drop to the bottom):
0 0 0 0
1 2 3 0
3 1 4 0
1 1 3 0

rescan: no row or column has 3+ equal, non-zero values -> stable, done
```

Intuition (diagram)

```
mark: scan every row & col, tag runs >= 3 (don't zero yet)
[1][2][3][4]      [ ][ ][ ][4*]
[2*][2*][2*][4*]  -> [2*][2*][2*][4*]  (row run + col run marked)
[3][1][4][4*]      [ ][ ][ ][4*]
[1][1][3][4*]      [ ][ ][ ][4*]
crush: zero every marked cell, then drop each column down
```

Approach (simulation)

Each round has three clean phases: **mark** every cell that belongs to a horizontal or vertical run of 3+ (using the candy's sign, so a cell already marked doesn't get miscounted by an overlapping run), **crush** the marked cells to 0, then **apply gravity** column by column. If a round marks nothing, the board is stable and we stop.

```

public int[][] candyCrush(int[][] board) {
    int rows = board.length, cols = board[0].length;
    boolean stable = false;

    while (!stable) {
        stable = true;

        // phase 1a: mark horizontal runs of 3+ (negate, don't zero yet)
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c + 2 < cols; c++) {
                int val = Math.abs(board[r][c]);
                if (val != 0 && val == Math.abs(board[r][c + 1])
                    && val == Math.abs(board[r][c + 2])) {
                    board[r][c] = board[r][c + 1] = board[r][c + 2] = -val;
                    stable = false;
                }
            }
        }

        // phase 1b: mark vertical runs of 3+ (same trick)
        for (int c = 0; c < cols; c++) {
            for (int r = 0; r + 2 < rows; r++) {
                int val = Math.abs(board[r][c]);
                if (val != 0 && val == Math.abs(board[r + 1][c])
                    && val == Math.abs(board[r + 2][c])) {
                    board[r][c] = board[r + 1][c] = board[r + 2][c] = -val;
                    stable = false;
                }
            }
        }

        if (stable) break; // nothing marked this round: done

        // phase 2 + 3: crush (zero) marked cells, then gravity per column
        for (int c = 0; c < cols; c++) {
            int write = rows - 1; // write pointer, bottom of column
            for (int r = rows - 1; r >= 0; r--) {
                if (board[r][c] > 0) { // survivor (unmarked, non-zero)
                    board[write--][c] = board[r][c];
                }
            }
            while (write >= 0) { // fill the rest of the column with 0
                board[write--][c] = 0;
            }
        }
    }
    return board;
}

```

Version	Time	Space
Simulation (mark, crush, drop, repeat)	$O((m \cdot n)^2)$ worst case	$O(1)$ extra (in place)

Each round costs $O(m \cdot n)$ to scan and crush, and a round always crushes at least 3 cells, so there are at most $O(m \cdot n)$ rounds before nothing is left to crush — hence the squared bound. In practice boards stabilize in a handful of rounds.

If the interviewer pushes for speed: only re-scan rows/columns that changed in the previous round (track a dirty set of touched rows/cols) instead of rescanning the whole board every round — same worst case, much faster on boards that settle quickly.

What to say

"This is pure grid simulation, not a search problem — no decision tree, just repeat a fixed cycle: mark every cell that's part of a 3+ run horizontally or vertically, crush all marked cells to zero at once, then drop each column's survivors down with a bottom-up write pointer. I mark with a sign flip instead of zeroing immediately so overlapping horizontal and vertical runs on the same cell don't get double-counted or missed. I stop the first round that marks nothing — that's the stable board."

9. Design Snake Game

Evidence: [LEETCODE — LC 353, "Design Snake Game"]

How the interviewer poses it. "Design a Snake game that runs on a `width x height` board. The snake starts at `(0, 0)` with length 1. You're given a list of `food` positions in the order they appear. Implement `move(direction)`, called once per game tick with `"U"`, `"D"`, `"L"`, or `"R"`: if the snake's new head lands on the next food cell, the snake **grows by one** (score += 1) and that food is consumed; otherwise the snake just advances — head moves forward, tail cell is vacated. Return the current score after a valid move, or `-1` the moment the snake hits a wall or runs into its own body."

De-story it

Strip the arcade nouns:

```
nouns to ignore: "snake", "board", "food", "grows"
INPUT    : board dims (width, height), an ordered list of food
           coordinates, and a stream of move() calls each carrying
           one direction
OUTPUT   : the score after each valid move, or -1 on the first
           invalid move (wall or self-collision)
CONSTRAINT: the snake's body is an ordered path of cells that must
             stay inside the board and never overlap itself — except
             the tail cell, which is vacated on the very same tick
             the head could move into it
```

That's a **stateful design problem**: one object, repeated `move()` calls, and you must maintain enough state between calls to answer in $O(1)$. The body is an **ordered sequence** (you need to know which end is the tail so you can drop it) plus you need **$O(1)$ membership tests** (is the new head cell already occupied?). Ordered sequence + fast membership → a **deque for the path, a hashset for occupancy** — the same "queue + set" combo you'd reach for in an LRU cache or a BFS frontier, just applied to a body of cells instead of nodes.

Worked example

```
width = 3, height = 2
food = [[1,2], [0,1]]          (in the order they must be eaten)
moves = "R", "D", "R", "U", "L", "U"

start: body = [(0,0)]          score 0

move R: head -> (0,1). not food[0]=(1,2). tail (0,0) drops.
        body = [(0,1)]          return 0
move D: head -> (1,1). not food[0]=(1,2). tail (0,1) drops.
        body = [(1,1)]          return 0
move R: head -> (1,2). == food[0]! grow, don't drop tail.
        body = [(1,2),(1,1)]    return 1
move U: head -> (0,2). not food[1]=(0,1). tail (1,1) drops.
        body = [(0,2),(1,2)]    return 1
move L: head -> (0,1). == food[1]! grow, don't drop tail.
        body = [(0,1),(0,2),(1,2)] return 2
move U: head -> (-1,1). off the board.          return -1
```

Intuition (diagram)

```
body deque: head -> [1,1] [1,0] [0,0] [0,1] <- tail
move "U": new head lands on (0,1) == old tail cell
tail is popped THIS move -> that cell is free -> legal!
```

Picture the four cells above as a tight loop: $(1,1) \rightarrow (1,0) \rightarrow (0,0) \rightarrow (0,1) \rightarrow (1,1)$. If the head moves "U" toward $(0,1)$, that's exactly where the tail currently sits — but the tail is leaving that cell on this same tick, so the move is perfectly legal. This is the **classic bug**: check the new head against the occupancy set only *after* the tail has been removed from it, otherwise a snake chasing its own tail around a loop will incorrectly report game over on every lap.

Approach (queue + hashset simulation)

```
class SnakeGame {
    private final int width, height;
    private final int[][] food;
    private int foodIndex = 0;
    private int score = 0;

    // deque of encoded cells (row * width + col); front = head, back = tail
    private final Deque<Integer> body = new LinkedList<>();
    private final Set<Integer> occupied = new HashSet<>();

    public SnakeGame(int width, int height, int[][] food) {
        this.width = width;
        this.height = height;
        this.food = food;
        body.addFirst(0);           // snake starts at (0,0)
        occupied.add(0);
    }

    public int move(String direction) {
        int head = body.peekFirst();
        int row = head / width;
        int col = head % width;
        switch (direction) {
            case "U": row--; break;
            case "D": row++; break;
            case "L": col--; break;
            case "R": col++; break;
        }
        if (row < 0 || row >= height || col < 0 || col >= width) {
            return -1;                // ran off the board
        }

        int newHead = row * width + col;
        boolean ateFood = foodIndex < food.length
            && food[foodIndex][0] == row
            && food[foodIndex][1] == col;

        if (!ateFood) {
            // vacate the tail BEFORE checking collision: moving into the
            // cell the tail is leaving this same tick is legal, not a crash
            int tail = body.removeLast();
            occupied.remove(tail);
        }

        if (occupied.contains(newHead)) {
            return -1;                // ran into the (still-occupied) body
        }

        body.addFirst(newHead);
        occupied.add(newHead);

        if (ateFood) {
            score++;
            foodIndex++;                // don't remove tail -> snake grows
        }
        return score;
    }
}
```

```
}  
}
```

Operation	Time	Space
<code>move()</code>	$O(1)$ — one deque push/pop pair, one hashset add/remove/contains	$O(1)$ extra per call
Overall	$O(1)$ per move	$O(n)$, where n = current snake length (body deque + hashset)

What to say

"This is a design problem, so I need state that survives across calls, not a one-shot algorithm. The snake body is an ordered path — I need to know which end is the tail so I can drop it — so I'll keep it in a deque, head at the front. I also need $O(1)$ membership checks for collision, so I'll mirror the same cells in a hashset. On each move I compute the new head, bail out on a wall hit, and — if no food was eaten — pop the tail out of *both* structures before checking the new head against the set. That ordering matters: the tail cell is vacated the same tick the head could move into it, so checking collision before popping the tail would wrongly flag a snake chasing its own tail as game over. If food is eaten, I skip the tail pop entirely so the snake grows by one."

10. Sliding Puzzle

Evidence: [LEETCODE — LC 773, "Sliding Puzzle"]

How the interviewer poses it. "You have a 2x3 board with tiles numbered 1 through 5 and one empty slot marked 0. A move slides the empty slot into an **adjacent** tile (up/down/left/right), swapping the two. Given the starting board, return the **minimum number of moves** needed to reach the solved board `[[1,2,3],[4,5,0]]`. If it's impossible, return `-1`."

De-story it

Strip the puzzle nouns:

```
nouns to ignore: "board", "tiles", "slides", "empty slot"
INPUT    : a 2x3 grid of the digits 0-5, each appearing once
OUTPUT   : the minimum number of swaps to reach a specific target grid
CONSTRAINT: a swap only exchanges 0 with one of its (up to 4) neighbors
```

That is a **shortest-path-in-an-unweighted-graph** problem where the "nodes" aren't numbers or grid cells — **each node is an entire board configuration**. Moving one tile creates a new node (a new board). Minimum moves between two configurations = **BFS from the start configuration to the target configuration**.

Worked example

```
start:  [[1,2,3],
         [4,0,5]]
target: [[1,2,3],
         [4,5,0]]

move 1: swap 0 with 5 (0 slides right)
        [[1,2,3],
         [4,5,0]]  <-- matches target

answer: 1
```

Intuition (diagram)

The key mental shift: don't think "move a tile," think "the whole board is one state, and each legal swap draws an edge to another whole-board state."

[1,2,3]	swap 0<->5	[1,2,3]	
[4,0,5]	----->	[4,5,0]	(target! distance = 1)
[1,2,3]	swap 0<->2	[1,0,3]	
[4,0,5]	----->	[4,2,5]	(a different state, distance = 1)

Every board is a node; every legal swap is an edge of weight 1. BFS layer-by-layer from the start node finds the shortest edge-count to the target node — exactly like BFS on a grid, except the "grid" here is the space of all $6!$ board permutations.

Approach (BFS over board states)

Flatten the 2x3 board into a length-6 string, e.g. `[[1,2,3],[4,0,5]] -> "123405"`. That string is the state. Precompute, for each flattened index, which other indices are adjacent on the 2x3 board — that map is fixed and reusable for every state:

flattened index layout:	neighbor map (for 2x3, row-major):
0 1 2	0: [1, 3]
3 4 5	1: [0, 2, 4]
	2: [1, 5]
	3: [0, 4]
	4: [1, 3, 5]
	5: [2, 4]

BFS: from each state, find the index of `'0'`, swap it with every neighbor index to generate the next states, and stop the first time you produce the target string.

```

public int slidingPuzzle(int[][] board) {
    String target = "123450";
    StringBuilder sb = new StringBuilder();
    for (int[] row : board) for (int v : row) sb.append(v);
    String start = sb.toString();

    int[][] neighbors = {
        {1, 3}, {0, 2, 4}, {1, 5},
        {0, 4}, {1, 3, 5}, {2, 4}
    };

    Set<String> visited = new HashSet<>();
    Queue<String> queue = new LinkedList<>();
    visited.add(start);
    queue.offer(start);
    int moves = 0;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        for (int i = 0; i < levelSize; i++) {
            String cur = queue.poll();
            if (cur.equals(target)) return moves;    // whole-board match

            int zero = cur.indexOf('0');
            for (int next : neighbors[zero]) {
                char[] chars = cur.toCharArray();
                char tmp = chars[zero];
                chars[zero] = chars[next];
                chars[next] = tmp;
                String swapped = new String(chars);
                if (visited.add(swapped)) {          // add() returns false if already present
                    queue.offer(swapped);
                }
            }
            moves++;
        }
    }
    return -1;    // target unreachable
}

```

Version	Time	Space
BFS over board states	$O((m \cdot n)!)^2$ states worst case — for 2x3, $6! = 720$	$O((m \cdot n)!)$ for visited set + queue

Why it's tiny in practice: for a 2x3 board there are at most $6! = 720$ distinct configurations, and only half are reachable from any given start (moves alternate parity), so BFS explores at most a few hundred states — effectively instant.

What to say

"Once I strip the tile-sliding story, the target isn't a cell or a number — it's an entire board. So each board configuration is a node, and a legal swap is an edge to a neighboring node. Minimum moves is shortest path in that graph, which is BFS: encode each board as a string for the visited set, precompute which flattened positions are adjacent, and expand by swapping 0 with each neighbor until I hit the target string or exhaust the queue."

11. Open the Lock

Evidence: [LEETCODE — LC 752, "Open the Lock"]

How the interviewer poses it. "You have a lock with 4 circular wheels. Each wheel has the digits 0 through 9, and it wraps around — so from 9 one turn forward lands on 0, and from 0 one turn backward lands on 9. The lock starts at "0000". You're given a list of deadends: combinations that, if the lock ever displays them, cause it to jam forever — you must never dial into one, not even as a stop along the way. You're also given a target combination. Each move turns exactly one wheel one step in one direction. Return the **minimum number of turns** required to reach target, or -1 if it's impossible."

De-story it

Strip the lock-and-dial nouns:

```
nouns to ignore: "wheel", "dial", "turn", "lock"
INPUT    : a start string "0000", a target string of 4 digits,
           and a set of forbidden strings (deadends)
OUTPUT   : the fewest single-character edits (each edit is +1 or -1,
           mod 10, on exactly one position) to turn start into target
CONSTRAINT: never visit a forbidden string, including the start
           itself if it happens to be a deadend
```

That is **shortest path in an unweighted graph** where each 4-digit string is a node and an edge connects two strings that differ by one wheel-turn on exactly one digit. Unweighted shortest path from a single source → **BFS**, not Dijkstra, not DFS. Deadends are simply nodes you delete from the graph before you start (mark them visited up front so BFS never enqueues them).

Worked example

```
deadends = ["0201","0101","0102","1212","2002"]
target   = "0202"

BFS layers from "0000":
  layer 0: 0000
  layer 1: 1000,9000,0100,0900,0010,0090,0001,0009 (8 neighbors)
  layer 2: ... skip 0100->0101/0102? not needed, "0101"/"0102" are
           only reachable if 0100 is visited; deadends just block
           those specific strings if BFS reaches them
  ...
  layer 6: 0202 found <-- answer: 6
```

Intuition (diagram)

Each of the 4 wheels can move +1 or -1 (wrapping at 0/9), so every state has up to 8 neighbors. BFS explores states ring-by-ring around "0000":

```
wheel positions:      [w0][w1][w2][w3]   e.g. "0 0 0 0"
one move = +-1 on exactly one wheel, wraps 9<->0

      0000                      (start, layer 0)
     /  |  \   ... (8 neighbors, one per wheel/direction)
    1000 9000 0100 ...         (layer 1)
     |    |    |
    2000 8000 0200 ...         (layer 2, closer to target)
     ...
target "0202" appears at the layer = min turns
deadends are simply erased from this graph before BFS starts
```

Approach (BFS + dead-ends)

Mark every deadend (and the visited set) before the first move, then run plain BFS layer by layer, generating the 8 neighbors of each state with wraparound arithmetic.


```

public int openLock(String[] deadends, String target) {
    Set<String> visited = new HashSet<>(Arrays.asList(deadends));
    if (visited.contains("0000")) return -1;    // blocked at the start
    if (target.equals("0000")) return 0;

    Queue<String> queue = new LinkedList<>();
    queue.offer("0000");
    visited.add("0000");

    int turns = 0;
    while (!queue.isEmpty()) {
        int size = queue.size();           // process one full layer
        turns++;
        for (int i = 0; i < size; i++) {
            String cur = queue.poll();
            for (String next : neighbors(cur)) {
                if (visited.contains(next)) continue;    // deadend or seen
                if (next.equals(target)) return turns;
                visited.add(next);
                queue.offer(next);
            }
        }
    }
    return -1;    // target unreachable without hitting a deadend
}

private List<String> neighbors(String s) {
    List<String> result = new ArrayList<>(8);
    char[] chars = s.toCharArray();
    for (int wheel = 0; wheel < 4; wheel++) {
        char original = chars[wheel];

        chars[wheel] = original == '9' ? '0' : (char) (original + 1);    // turn +1, wraps 9->0
        result.add(new String(chars));

        chars[wheel] = original == '0' ? '9' : (char) (original - 1);    // turn -1, wraps 0->9
        result.add(new String(chars));

        chars[wheel] = original;    // restore before touching the next wheel
    }
    return result;
}

```

If the interviewer pushes for faster: run **bidirectional BFS** — expand simultaneously from "0000" and from target, always growing whichever frontier is smaller, and stop when the frontiers meet. Same correctness, but it shrinks the explored state space from roughly 10^4 to roughly $2 \cdot \sqrt{10^4}$ in the typical case.

Version	Time	Space
BFS from source	$O(10^4 \cdot 8)$ states/edges explored	$O(10^4)$ visited set + queue
Bidirectional BFS	$O(10^4 \cdot 8)$ worst case, far less in practice	$O(10^4)$ two visited sets

What to say

"Ignoring the lock framing, this is unweighted shortest path on a graph of 4-digit strings, where an edge is a ± 1 turn on one wheel with wraparound. Unweighted shortest path from a single source means BFS. I pre-mark deadends as visited so BFS never enqueues them, expand each state into its 8 neighbors, and return the layer at which I first dequeue the target, or -1 if the queue empties first. If the interviewer wants it faster, I'd switch to bidirectional BFS, meeting in the middle from both '0000' and the target."

12. Stone Game

Evidence: [LEETCODE — LC 877, "Stone Game"]

How the interviewer poses it. "Alex and Lee play a game with piles of stones, `piles`. There are always an **even number of piles**, and the total number of stones is **odd** (so there can never be a tie). The players alternate turns, Alex going first. On each turn, a player takes the **entire pile** from either the **left end or the right end** of the row of remaining piles, and that pile's stones are added to their score. Both players play **optimally**, each trying to end up with more stones than the other. Return `true` if Alex (Player 1) can guarantee a win."

De-story it

Strip the tabletop-game nouns:

```
nouns to ignore: "Alex", "Lee", "turn", "pile of stones"
INPUT    : an array of positive integers piles[0..n-1], n even,
          sum(piles) odd
OUTPUT   : true if the FIRST mover can force
          (their total) > (the other mover's total),
          assuming both sides always choose optimally
CONSTRAINT: each move removes the pile at the LEFT end or the
          RIGHT end of whatever remains – no other removals
          are legal
```

That is the same shape you should recognize instantly: a **minimax / optimal-adversary** problem over a shrinking interval, where at every step the current player has exactly two choices (take `piles[i]` or take `piles[j]`) and the opponent then plays the same way on whatever remains. "Both play optimally" plus "either end of a row" is the signature of **interval DP with a minimax recurrence** — not a greedy "always take the bigger end" pick, which fails on cases where taking the smaller end now sets up a bigger end later.

Worked example

```
piles = [5, 3, 4, 5]          (n=4 even, sum=17 odd)

dp[i][j] = best SCORE DIFFERENCE (current mover minus opponent)
           achievable on the sub-array piles[i..j]

base cases (single pile, i == j):
    dp[0][0]=5  dp[1][1]=3  dp[2][2]=4  dp[3][3]=5

length 2:
    dp[0][1] = max(5 - dp[1][1], 3 - dp[0][0]) = max(2, -2) = 2
    dp[1][2] = max(3 - dp[2][2], 4 - dp[1][1]) = max(-1, 1) = 1
    dp[2][3] = max(4 - dp[3][3], 5 - dp[2][2]) = max(-1, 1) = 1

length 3:
    dp[0][2] = max(5 - dp[1][2], 4 - dp[0][1]) = max(4, 2) = 4
    dp[1][3] = max(3 - dp[2][3], 5 - dp[1][2]) = max(2, 4) = 4

length 4:
    dp[0][3] = max(5 - dp[1][3], 5 - dp[0][2]) = max(1, 1) = 1

dp[0][3] = 1 > 0  ->  Alex wins  ->  return true
```

Intuition (diagram)

At each step the mover takes `piles[i]` or `piles[j]`. Whichever end is taken, the OPPONENT becomes the "current mover" on what's left, so their best difference gets SUBTRACTED, not added.

```
row [ i ..... j ]
    take i: gain piles[i], opponent then owns [i+1..j]
            -> net = piles[i] - dp[i+1][j]
    take j: gain piles[j], opponent then owns [i..j-1]
            -> net = piles[j] - dp[i][j-1]
dp[i][j] = max(piles[i]-dp[i+1][j], piles[j]-dp[i][j-1])
base case: dp[i][i] = piles[i]          (only one pile left)
answer   : dp[0][n-1] > 0  =>  Alex forces a strict win
```

This is the **general-purpose tool**: exactly the recurrence you'd reach for on Problem 13 (Predict the Winner), which is the same "take from either end" game without LC 877's special guarantees. It's worth knowing that LC 877's specific constraints (even pile count, odd total, strict win required) hide a **cute O(1) shortcut**: index the piles `0, 1, 2, ...` and notice Alex can always commit up front to "I'll only ever take from the even-indexed positions" or "I'll only ever take from the odd-indexed positions" — one of those two groups sums to strictly more than the other (the total is odd, so they can't tie), and Alex can force taking every pile in whichever group is bigger no matter how Lee plays. So for LC 877 specifically, **the answer is always true** and you could return it in O(1). Say that out loud to show you spotted it, but then build the DP anyway — the DP is what still works the moment those guarantees vanish, which is exactly what happens in Problem 13.

Brute force

Recurse on both choices at every step, tracking the score difference directly — no memoization, so the same (i, j) interval gets recomputed many times.

```
public boolean stoneGameBrute(int[] piles) {
    return scoreDiff(piles, 0, piles.length - 1) > 0;
}

// returns (current mover's total) - (opponent's total) for piles[i..j],
// assuming both play optimally from here on
private int scoreDiff(int[] piles, int i, int j) {
    if (i == j) return piles[i]; // only one pile left to take
    int takeLeft = piles[i] - scoreDiff(piles, i + 1, j);
    int takeRight = piles[j] - scoreDiff(piles, i, j - 1);
    return Math.max(takeLeft, takeRight); // current mover picks the better end
}
```

Optimized (bottom-up interval DP)

Fill $dp[i][j]$ by increasing interval length so $dp[i+1][j]$ and $dp[i][j-1]$ are already known before $dp[i][j]$ needs them — this eliminates the repeated recursive calls above.

```
public boolean stoneGame(int[] piles) {
    int n = piles.length;
    if (n <= 1) return true; // nothing to contest

    int[][] dp = new int[n][n];
    for (int i = 0; i < n; i++) dp[i][i] = piles[i];

    for (int len = 2; len <= n; len++) { // grow the interval width
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            int takeLeft = piles[i] - dp[i + 1][j];
            int takeRight = piles[j] - dp[i][j - 1];
            dp[i][j] = Math.max(takeLeft, takeRight);
        }
    }
    return dp[0][n - 1] > 0;
}
```

Because $dp[i][j]$ only ever reads row $i+1$ and row i , this compresses to a **1D rolling array**, cutting space to $O(n)$ — mention it if pushed on space, but write the 2D version first.

Version	Time	Space
Brute force (recursion, no memo)	$O(2^n)$	$O(n)$ recursion
Bottom-up interval DP	$O(n^2)$	$O(n^2)$ ($O(n)$ with rolling array)
LC 877's $O(1)$ parity shortcut	$O(1)$	$O(1)$

What to say

"Both play optimally' plus 'take from either end' is minimax, not greedy. I define `dp[i][j]` as the best SCORE DIFFERENCE the current mover can force on `piles[i..j]`: take the left end and subtract whatever the opponent then forces on `dp[i+1][j]`, or take the right end and subtract `dp[i][j-1]`, and pick whichever is larger. Alex wins iff `dp[0][n-1] > 0`. I can build that bottom-up by interval length in $O(n^2)$. I'll also flag that LC 877's specific guarantees — even pile count, odd total — mean Alex can always force a win in $O(1)$ just by claiming all even-indexed or all odd-indexed piles, whichever group sums higher; but I'd still reach for the DP first, because it's the version that keeps working on Problem 13 (Predict the Winner), where that parity shortcut no longer applies."

13. Predict the Winner

Evidence: [LEETCODE — LC 486, "Predict the Winner"]

How the interviewer poses it. "Two players take turns picking numbers from an array, `nums`. On each turn, a player may take the number at **either end** of the remaining array and add it to their own running score. Both players play **optimally** — each one is trying to maximize their own final score. Player 1 moves first. Given `nums`, return `true` if Player 1 can guarantee a score that is **greater than or equal to** Player 2's score, and `false` otherwise."

De-story it

Strip the game-show nouns:

```
nouns to ignore: "player", "turn", "pick a number"
INPUT   : an array of integers nums[0..n-1]
OUTPUT  : true if the FIRST mover can force
          (their total) >= (the other mover's total),
          assuming both sides always choose optimally
CONSTRAINT: each move removes an element from the LEFT end
              or the RIGHT end of whatever remains – no other
              removals are legal
```

That is a **minimax / optimal-adversary** problem over a shrinking interval: at every step the current player has exactly two choices (take `nums[i]` or take `nums[j]`), and the opponent then plays the same way on what's left. Whenever the phrase "both play optimally" shows up alongside "either end of a range," that's the signature of **interval DP with a minimax recurrence**, not a greedy pick.

Worked example

```
nums = [1, 5, 233, 7]

dp[i][j] = best SCORE DIFFERENCE (current mover minus opponent)
          achievable on the sub-array nums[i..j]

base cases (single element, i == j):
    dp[0][0]=1  dp[1][1]=5  dp[2][2]=233  dp[3][3]=7

length 2:
    dp[0][1] = max(1 - dp[1][1], 5 - dp[0][0]) = max(1-5, 5-1) = 4
    dp[1][2] = max(5 - dp[2][2], 233 - dp[1][1]) = max(-228, 228) = 228
    dp[2][3] = max(233 - dp[3][3], 7 - dp[2][2]) = max(226, -226) = 226

length 3:
    dp[0][2] = max(1 - dp[1][2], 233 - dp[0][1]) = max(1-228, 233-4) = 229
    dp[1][3] = max(5 - dp[2][3], 7 - dp[1][2]) = max(5-226, 7-228) = -221

length 4:
    dp[0][3] = max(1 - dp[1][3], 7 - dp[0][2]) = max(1-(-221), 7-229) = 222

dp[0][3] = 222 >= 0  ->  Player 1 wins  ->  return true
```

Intuition (diagram)

At each step the mover picks `nums[i]` or `nums[j]`. Whichever end is taken, the OPPONENT then becomes the "current mover" on what's left, so their best difference gets SUBTRACTED, not added.

```
range [ i ..... j ]
  take i: gain nums[i], opponent then owns [i+1..j]
          -> net = nums[i] - dp[i+1][j]
  take j: gain nums[j], opponent then owns [i..j-1]
          -> net = nums[j] - dp[i][j-1]
dp[i][j] = max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1])
base case: dp[i][i] = nums[i]          (only one item left)
answer   : dp[0][n-1] >= 0 => Player 1 can force a tie or win
```

This is the **same shape** as Problem 12 (Stone Game): score-difference interval DP over "pick from either end." Stone Game can skip the DP entirely because parity math *guarantees* Player 1 always wins for that setup; Predict the Winner has no such guarantee, so you must actually run the recurrence to find out which side comes out ahead.

Brute force

Recurse on both choices at every step, tracking whose turn it is and each player's running total directly — no memoization, so the same `(i, j)` interval gets recomputed many times.


```

public boolean predictTheWinnerBrute(int[] nums) {
    return scoreDiff(nums, 0, nums.length - 1) >= 0;
}

// returns (current mover's total) - (opponent's total) for nums[i..j],
// assuming both play optimally from here on
private int scoreDiff(int[] nums, int i, int j) {
    if (i == j) return nums[i]; // only one item left to take
    int takeLeft = nums[i] - scoreDiff(nums, i + 1, j);
    int takeRight = nums[j] - scoreDiff(nums, i, j - 1);
    return Math.max(takeLeft, takeRight); // current mover picks the better end
}

```

Optimized (bottom-up interval DP)

Fill `dp[i][j]` by increasing interval length so `dp[i+1][j]` and `dp[i][j-1]` are already known before `dp[i][j]` needs them — this eliminates the repeated recursive calls above.

```

public boolean predictTheWinner(int[] nums) {
    int n = nums.length;
    if (n <= 1) return true; // nothing to contest

    int[][] dp = new int[n][n];
    for (int i = 0; i < n; i++) dp[i][i] = nums[i];

    for (int len = 2; len <= n; len++) { // grow the interval width
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            int takeLeft = nums[i] - dp[i + 1][j];
            int takeRight = nums[j] - dp[i][j - 1];
            dp[i][j] = Math.max(takeLeft, takeRight);
        }
    }
    return dp[0][n - 1] >= 0;
}

```

Because `dp[i][j]` only ever reads row `i+1` and row `i`, you can compress this to a **1D rolling array** and cut space to $O(n)$ — worth mentioning if the interviewer asks for tighter space, but the 2D version above is the one to write first.

Version	Time	Space
Brute force (recursion, no memo)	$O(2^n)$	$O(n)$ recursion
Bottom-up interval DP	$O(n^2)$	$O(n^2)$ ($O(n)$ with rolling array)

What to say

"Both play optimally' plus 'take from either end' means I can't just simulate one path — I need minimax. I define `dp[i][j]` as the best SCORE DIFFERENCE the current mover can force on `nums[i..j]` : they either take the left end and subtract whatever the opponent then forces on `dp[i+1][j]`, or take the right end and subtract `dp[i][j-1]`, and they pick whichever is larger. Player 1 wins or ties exactly when `dp[0][n-1] >= 0`. It's the same score-difference interval DP as Stone Game, except here there's no shortcut parity

argument, so I build the table bottom-up by interval length in $O(n^2)$, and I'd mention the $O(n)$ rolling-array compression if pushed on space."

14. Frog Jump

Evidence: [LEETCODE — LC 403, "Frog Jump" (Hard)]

How the interviewer poses it. "A frog is crossing a river. The river is divided into units, and at each unit there may or may not be a stone. The frog can jump on a stone, but not into the water. Given a list of `stones` positions (in strictly increasing order), determine if the frog can cross the river by landing on the last stone. The frog starts on the first stone and assumes the **first jump must be 1 unit**. If the frog's last jump was `k` units, its next jump must be `k - 1`, `k`, or `k + 1` units, and it must always be a **positive** number of units."

De-story it

Strip the river/frog nouns:

```
nouns to ignore: "frog", "river", "stone", "jump"
INPUT    : a strictly increasing array of positions (the stones)
OUTPUT   : true/false – can you reach the last index starting from
           index 0, where each "step size" (k) constrains the next
           step size to k-1, k, or k+1, and step size starts at 1?
CONSTRAINT: step size must stay > 0; you may only land on a
           position that exists in the array
```

That is a **reachability search over an expanded state space**. The naive instinct is to treat this like a graph problem over *positions* — but position alone doesn't capture enough information: from the same stone, arriving with a jump of size 3 unlocks different next moves than arriving with a jump of size 7. **The state is (position, lastJumpSize), not just position**. Once you see that, it's a standard memoized-search / DP problem over that expanded state.

Worked example

```
stones = [0, 1, 3, 5, 6, 8, 12, 17]
index   0  1  2  3  4  5  6  7

start: stone 0, first jump must be size 1 -> lands on stone 1 (pos 1)
from stone 1, last jump k=1 -> try k-1=0(invalid), k=1, k+1=2
  jump 2 -> pos 3 (stone 2)    k=2
from stone 2 (k=2) -> try 1, 2, 3
  jump 2 -> pos 5 (stone 3)    k=2
from stone 3 (k=2) -> try 1, 2, 3
  jump 1 -> pos 6 (stone 4)    k=1
from stone 4 (k=1) -> try 1, 2 (0 invalid)
  jump 2 -> pos 8 (stone 5)    k=2
from stone 5 (k=2) -> try 1, 2, 3, 4
  jump 4 -> pos 12 (stone 6)   k=4
from stone 6 (k=4) -> try 3, 4, 5
  jump 5 -> pos 17 (stone 7)   k=5 <--- last stone, return true
```

Intuition (diagram)

pos:	0	1	3	5	6	8	12	17
	S0	S1	S2	S3	S4	S5	S6	S7
jump:		1	2	2	1	2	4	5
last k:		1	2	2	1	2	4	5

next k in {k-1,k,k+1}, must stay > 0, must land on a stone

Brute force

Depth-first search: from the current stone with the last jump size `k`, try every candidate next jump (`k-1`, `k`, `k+1`), checking (via a set or binary search) whether the resulting position is an actual stone. No memoization, so the same `(position, k)` pair can be re-explored many times — the branching factor is up to 3 per hop.

```
public boolean canCrossBrute(int[] stones) {
    Set<Integer> positions = new HashSet<>();
    for (int s : stones) positions.add(s);
    if (stones.length == 1) return true;
    return dfs(positions, stones[stones.length - 1], stones[0], 0);
}

private boolean dfs(Set<Integer> positions, int target, int pos, int lastJump) {
    if (pos == target) return true;
    for (int k = lastJump - 1; k <= lastJump + 1; k++) {
        if (k <= 0) continue;           // jump size must be positive
        int next = pos + k;
        if (positions.contains(next) && dfs(positions, target, next, k)) {
            return true;               // note: lastJump starts at 0
        }                             // so first real jump is 1
    }
    return false;
}
```

Optimized (forward DP over jump-size sets)

Same insight — state is `(position, lastJumpSize)` — but instead of re-deriving reachable states from scratch, build a **map from each stone to the set of jump sizes that can reach it**, filled in left to right. Every stone is visited once, and every jump size at a stone is expanded at most once, which removes the exponential re-exploration.

```

public boolean canCross(int[] stones) {
    int n = stones.length;
    if (stones[1] != 1) return false;           // first jump must be size 1

    Map<Integer, Set<Integer>> jumpsAtStone = new HashMap<>();
    for (int s : stones) jumpsAtStone.put(s, new HashSet<>());
    jumpsAtStone.get(stones[0]).add(0);         // "arrived" at stone 0 with k=0

    for (int stone : stones) {
        for (int k : jumpsAtStone.get(stone)) {
            for (int next = k - 1; next <= k + 1; next++) {
                if (next <= 0) continue;         // jump size must stay positive
                int landing = stone + next;
                if (jumpsAtStone.containsKey(landing)) {
                    jumpsAtStone.get(landing).add(next); // reachable with size `next`
                }
            }
        }
    }
    return !jumpsAtStone.get(stones[n - 1]).isEmpty();
}

```

Version	Time	Space
Brute force DFS (no memo)	$O(3^n)$ worst case	$O(n)$ recursion
Forward DP (jump-size sets)	$O(n^2)$	$O(n^2)$ worst case (sets of jump sizes)

If the interviewer asks for the memoized-DFS variant instead: cache on the pair `(stoneIndex, lastJumpSize)` with a `HashMap<Long, Boolean>` (pack the two ints into one long key) — it's the same $O(n^2)$ bound, just top-down instead of bottom-up.

What to say

"Stripped of the river framing, this is reachability search — but the state isn't just 'which stone am I on,' it's 'which stone am I on *and* what was my last jump size,' because that pair determines every future move. I'll build, for each stone, the set of jump sizes that can land on it, filling it in left to right: for every jump size at a stone, try size-1, size, and size+1 on the next landing spot. That's $O(n^2)$ instead of the exponential brute-force DFS, and the last stone's set being non-empty is my answer."

15. Ability Cooldown Scheduler

Evidence: [LEETCODE — LC 621, "Task Scheduler" (game-framed variant)]

How the interviewer poses it. "Your game character has a list of `abilities` queued to cast, given as an array where each entry is the **ability type** (e.g. `'F'` for fireball, `'H'` for heal). Casting the **same ability type twice** requires `n` **turns of cooldown** in between — you cannot recast it until the cooldown expires. On any turn you either cast an ability that's off cooldown, or **idle** if nothing is available. Return the **minimum number of turns** needed to cast every ability in the list, in some order."

De-story it

Strip the fantasy nouns:

```
nouns to ignore: "character", "cast", "ability", "cooldown turns"
INPUT    : a list of symbols (ability types) and an integer n (the
           required gap between two uses of the same symbol)
OUTPUT   : the minimum number of slots (turns) to place every symbol,
           inserting idle slots where no symbol is available
CONSTRAINT: identical symbols must be at least (n + 1) slots apart
```

That is **task scheduling with cooldown**: place items into a timeline so that repeats of the same item are spaced out by at least `n`, filling unavoidable gaps with idle slots. The **most frequent symbol sets the lower bound** on the schedule — everything else just fills in around it → **greedy, frequency-driven** problem, not search/DP.

Worked example

```
abilities = [F, F, F, H, H, S]    n = 2    (need 2-turn gap between repeats)
```

```
F appears 3x (max), H appears 2x, S appears 1x
maxFreq = 3, countMax = 1 (only F reaches 3)
```

one valid schedule:

```
F H S F H idle F      (S or idle can also share the slack)
1 2 3 4 5 6    7    <- 7 turns total
```

formula check:

```
(maxFreq - 1) * (n + 1) + countMax = (3-1)*(2+1) + 1 = 7
total abilities = 6
answer = max(6, 7) = 7    matches
```

Intuition (diagram)

Picture the **most frequent** ability as fixed anchors with `n`-wide gaps after each one (except the last). Every other ability just fills the open slots between anchors — if there aren't enough fillers, idle turns take

their place.

```
F . . F . . F    <- F fixed, gap = n = 2 after each (last has none)
F H S F H _ F    <- fillers (H,H,S) slot into the gaps; _ = idle
```

If `countMax` abilities tie for the max frequency, they each need their own column across those gap rows, which is why `countMax` is added once at the end (the last row is never padded with a trailing gap).

Approach

Simulate turn by turn: track each ability's next-available turn; each turn, cast the available ability with the highest remaining count (or idle if none are available). This is intuitive but pays a heap cost per turn.

```
public int leastIntervalHeap(char[] tasks, int n) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char t : tasks) freq.merge(t, 1, Integer::sum);

    PriorityQueue<Integer> heap = new PriorityQueue<>(Collections.reverseOrder());
    heap.addAll(freq.values()); // max-heap of remaining counts

    int turns = 0;
    while (!heap.isEmpty()) {
        List<Integer> cooling = new ArrayList<>(); // abilities used this round
        for (int slot = 0; slot <= n; slot++) { // one round = n+1 slots
            if (!heap.isEmpty()) {
                int cnt = heap.poll();
                if (cnt - 1 > 0) cooling.add(cnt - 1);
            }
            turns++;
            if (heap.isEmpty() && cooling.isEmpty()) break; // nothing left, stop early
        }
        heap.addAll(cooling); // abilities come off cooldown
    }
    return turns;
}
```

Optimized (math formula)

The cast counts alone determine the answer: the most frequent ability forces $(\text{maxFreq} - 1)$ gaps of width $(n + 1)$, plus one column per ability tied for that max frequency. If there are enough distinct abilities to fill every gap, idle turns vanish and the total is simply the number of abilities.

```
public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char t : tasks) freq[t - 'A']++;

    int maxFreq = 0;
    for (int f : freq) maxFreq = Math.max(maxFreq, f);

    int countMax = 0;
    for (int f : freq) if (f == maxFreq) countMax++;

    int gapsSchedule = (maxFreq - 1) * (n + 1) + countMax; // lower bound from the busiest ab
    return Math.max(tasks.length, gapsSchedule); // never fewer turns than tasks
}
```

Version	Time	Space
Heap simulation	$O(T \log 26)$	$O(26)$
Math formula (optimized)	$O(T)$	$O(26)$

(T = total number of abilities to cast; the alphabet of ability types is bounded, so the heap/array is effectively $O(1)$ space.)

What to say

"Ignoring the fantasy framing, this is task scheduling with a cooldown gap. The busiest ability type sets a lower bound: it needs $(\text{maxFreq}-1)$ gaps of width $n+1$, plus one slot per ability tied for that max — everything else just fills the gaps or, if there isn't enough to fill them, idle turns do. So the answer is $\max(\text{totalAbilities}, (\text{maxFreq}-1)*(n+1) + \text{countMax})$. I could also simulate with a max-heap picking the most-depleted-but-available ability each turn, but the frequency formula gets there in $O(T)$."

16. Design Leaderboard

Evidence: [LEETCODE — LC 1244, "Design A Leaderboard"]

How the interviewer poses it. "Design a `Leaderboard` for a game with three operations: `addScore(playerId, score)` adds `score` to that player's total (creating the player with that score if they're new), `top(K)` returns the **sum of the top K scores** on the board, and `reset(playerId)` removes that player from the leaderboard entirely. All three operations should be efficient even as players keep addingScore and calling top(K) repeatedly."

De-story it

Strip the game nouns:

```
nouns to ignore: "player", "game", "board"
INPUT    : a stream of two commands over a set of (id, value)
           pairs -- "add value to id's total" and "remove id
                 entirely" -- plus a query "sum of K largest values"
OUTPUT   : addScore/reset return nothing (mutate state); top(K)
           returns an integer, the sum of the K largest values
CONSTRAINT: values change over time and K varies per call, so the
           query must not re-sort everything from scratch each time
```

That is a **design problem needing two views of the same data**: fast **lookup by id** (for add/reset) and fast **access in sorted order** (for top). No single structure gives you both for free — you keep a `HashMap` for the lookup and a sorted structure for the ranking, and update both together.

Worked example

```
addScore(1, 73) // player 1: 73
addScore(2, 56) // player 2: 56
addScore(3, 39) // player 3: 39
addScore(4, 51) // player 4: 51
addScore(3, 4)  // player 3: 39 + 4 = 43
top(3)          // three highest: 73, 56, 51 -> 180
reset(1)         // player 1 leaves
reset(2)         // player 2 leaves
addScore(2, 51) // player 2 rejoins fresh with 51
top(3)          // remaining: 43, 51, 51 -> 145
```

Intuition (diagram)

```
playerId -> score (HashMap)      score -> count (TreeMap, sorted)
  2 -> 51                        43 -> 1
  3 -> 43                        51 -> 2    <- players 2 and 4 tie
  4 -> 51

top(3) walks the TreeMap from the highest key down:
  51 (count 2, take 2) -> sum = 102
  43 (count 1, take 1) -> sum = 145
  remaining players needed = 0 -> stop, return 145
```

The `HashMap` answers "what does this player have right now?" in $O(1)$. The `TreeMap` answers "what are the biggest values, in order?" by keeping scores as **keys**, sorted, with a count of how many players currently hold each score (scores are not unique, so it's a sorted multiset, not a sorted set).

Approach (design)

```
class Leaderboard {
    // playerId -> current total score
    private final Map<Integer, Integer> playerScore = new HashMap<>();
    // score -> how many players currently have that exact score
    private final TreeMap<Integer, Integer> scoreCount = new TreeMap<>();

    public void addScore(int playerId, int score) {
        int newScore = score;
        if (playerScore.containsKey(playerId)) {
            int oldScore = playerScore.get(playerId);
            newScore = oldScore + score;
            decrementScoreCount(oldScore);    // leaving the old bucket
        }
        playerScore.put(playerId, newScore);
        scoreCount.merge(newScore, 1, Integer::sum); // entering the new bucket
    }

    public int top(int K) {
        int sum = 0;
        int remaining = K;
        // walk buckets highest score first
        for (Map.Entry<Integer, Integer> e : scoreCount.descendingMap().entrySet()) {
            int score = e.getKey();
            int count = e.getValue();
            int take = Math.min(count, remaining);
            sum += take * score;
            remaining -= take;
            if (remaining == 0) break;        // collected K scores
        }
        return sum;
    }

    public void reset(int playerId) {
        int score = playerScore.remove(playerId);
        decrementScoreCount(score);
    }

    private void decrementScoreCount(int score) {
        int count = scoreCount.get(score);
        if (count == 1) {
            scoreCount.remove(score);        // last player at this score
        } else {
            scoreCount.put(score, count - 1);
        }
    }
}
```

Simpler-but-slower alternative: keep only the `HashMap` and, on every `top(K)` call, copy `playerScore.values()` into a list, sort it descending, and sum the first K. That's correct and easier to write under pressure, but it re-sorts on every single query. The `TreeMap` version amortizes the sorting cost across all the `addScore / reset` calls instead, which is what the interviewer is probing for by calling `top` "repeatedly."

Operation	HashMap-only (sort each time)	HashMap + TreeMap
addScore	$O(1)$	$O(\log n)$
reset	$O(1)$	$O(\log n)$
top(K)	$O(n \log n)$	$O(K + \log n)$

What to say

"This needs two views of the same data: $O(1)$ lookup by playerId for add/reset, and sorted access for top(K). A HashMap alone gives me the lookup but forces a full sort on every top(K) call. So I pair it with a TreeMap keyed by score, storing a count per score since scores can tie — that turns addScore and reset into a remove-from-old-bucket, add-to-new-bucket update, and top(K) into a walk down the TreeMap's highest keys until I've collected K players' worth of score."
